Chapter 5 Inheritance

CHAPTER OBJECTIVES

- Explore the derivation of new classes from existing ones.
- Define the concept and purpose of method overriding.
- Discuss the design of class hierarchies.
- Examine the purpose and use of abstract classes.
- Discuss the issue of visibility as it relates to inheritance.
- Discuss object-oriented design in the context of inheritance.

This chapter examines inheritance — one of the most fundamental and defining techniques in object-oriented programming. It is a deceptively simple idea with far-reaching implications for how we design and organize software, and for our ability to reuse existing classes across different contexts and programs. We explore how subclasses and class hierarchies are created, examine the technique of overriding inherited method definitions, introduce the `protected` visibility modifier, and analyze the effect of all visibility modifiers in the context of inheritance.

5.1 Creating Subclasses

In our introduction to object-oriented concepts in Chapter 1 we used the analogy that a class is to an object what a blueprint is to a house. That analogy has served us well: a class defines the characteristics and behaviors of a family of objects, while each object is a concrete realization of that definition. A class itself reserves no memory for instance variables — it is the plan, and objects are the embodiment of that plan.

Many houses can be built from the same blueprint. They are fundamentally the same house, differing only in location, interior furnishings, and the people who inhabit them. Now imagine wanting a house that is similar to an existing design but with certain additions or differences. Rather than starting from nothing, you begin with the original blueprint and modify it to accommodate the new requirements. This is exactly how many residential developments are built — a shared structural core gives all houses in the development their common layout, while specific variations — a fireplace here, a finished basement there, an upgraded kitchen in another — give each home its distinct character.

A master architect creates the foundational blueprint from which all the variant designs are derived. Each derived blueprint is simpler to create because the underlying structure is already established; only the distinguishing features need to be added. Creating a new blueprint by building on an existing one is directly analogous to the object-oriented concept of inheritance, in which a new class is derived from an existing one. Inheritance is one of the most powerful tools in the object-oriented programmer's toolkit, and it is a defining characteristic of the paradigm.

KEY CONCEPT
Inheritance is the process of deriving a new class from an existing one.

KEY CONCEPT
One purpose of inheritance is to reuse existing software.

Through inheritance, the derived class automatically acquires all of the variables and methods defined in the original class. The programmer can then extend or refine the derived class by adding new variables and methods, or by providing alternative definitions for inherited ones.

In general, creating new classes through inheritance is faster, easier, and less costly than writing them from scratch. Inheritance is a primary mechanism for software reuse — by building on software that has already been designed, implemented, and tested, we leverage all of that prior investment rather than duplicating it.

The concept of classification — grouping objects with shared characteristics — naturally gives rise to hierarchical structures. Consider biology: all mammals are warm-blooded and share many other characteristics. Horses are mammals, and therefore share all mammalian characteristics, but they also have unique features that distinguish them from other mammals such as dogs.

Translated into software terms: an existing class called `Mammal` would define the state and behavior common to all mammals. A `Horse` class derived from `Mammal` would automatically inherit everything `Mammal` defines, and could then introduce the additional variables and methods that distinguish horses from other mammals.

The original class from which a new class is derived is called the parent class, superclass, or base class. The newly derived class is called the child class or subclass. In UML, an inheritance relationship is depicted as an arrow with an open arrowhead pointing from the child class to the parent.

KEY CONCEPT
Inheritance creates an is-a relationship between the parent and child classes.

Inheritance should always establish a meaningful is-a relationship between the two classes. The child class should represent a more specific version of the parent. A horse is a mammal; not all mammals are horses, but every horse is unquestionably a mammal. For any class X derived from class Y, the statement "X is a Y" should be true and sensible. When it is not, inheritance is likely being misapplied.

Let's look at a concrete example. The `Words` program in Listing 5.1 creates an object of class `Dictionary`, which is derived from the class `Book`. Three methods are invoked through the `Dictionary` object: two defined directly within `Dictionary`, and one inherited from `Book`.

Java uses the reserved word `extends` to indicate that a class is being derived from an existing one. The `Book` class (Listing 5.2) serves as the parent of `Dictionary` (Listing 5.3)

simply by appearing in the `extends` clause of `Dictionary`'s class header. Through this relationship, `Dictionary` automatically inherits the `setPages` and `getPages` methods as well as the `pages` variable — they are available inside `Dictionary` as though they had been declared there directly. Notice that in the `Dictionary` class, the `computeRatio` method references the `pages` variable even though it was declared in `Book`.

Also note that while `Book` must exist for `Dictionary` to be defined, no `Book` object is ever instantiated in the program. An instance of a child class does not require an instance of the parent class to exist.

Inheritance operates in one direction only. The `Book` class has no access to variables or methods declared specifically in `Dictionary`. For example, an object created from `Book` cannot invoke `setDefinitions`. This restriction is logically sound: a dictionary has pages because all books have pages, but only a dictionary has definitions — that characteristic does not belong to books in general.

The `protected` Modifier

Visibility modifiers govern access to the members of a class, and they play a significant role in inheritance as well. Any `public` method or variable in a parent class can be directly referenced by name within the child class and through objects of the child class. Private members of the parent class, however, cannot be referenced in the child class at all — not even by name.

This creates a tension. Making a variable `public` so that a derived class can access it violates encapsulation. Java resolves this tension with a third visibility modifier: `protected`. A member declared `protected` can be accessed by any derived class, while still offering stronger encapsulation than `public` visibility. More precisely, a `protected` member is accessible by any class within the same package, as well as by subclasses anywhere. Notice that in the `Words` example, the `pages` variable in `Book` is declared `protected`, allowing the `Dictionary` class to reference it directly.

KEY CONCEPT
Protected visibility provides the best possible encapsulation that permits inheritance.

It is important to understand that the original visibility of a member is preserved when it is inherited. The `setPages` method, for instance, remains `public` in its inherited form within `Dictionary`.

A final point of clarification: all members of a parent class — even those declared `private` — are technically inherited by the child class. Their definitions exist and memory is reserved for their variables. The restriction is simply that they cannot be referenced by name. This distinction is explored further in section 5.4.

Constructors, however, are not inherited. A constructor is a specialized method for initializing a specific type of object, so it would make no sense for a class called `Dictionary` to possess a constructor named `Book`. Yet a child class may well need to trigger the parent's constructor — which is one of the primary uses of the `super` reference, described next.

The `super` Reference

The reserved word `super` allows a child class to refer to its parent class. Through `super`, the child class can access parent members that might otherwise be shadowed or simply need to be explicitly referenced.

One of the most common uses of `super` is to invoke the parent's constructor. Listing 5.4 shows a revised version of the `Words` program called `Words2`. It introduces `Book2` (Listing 5.5) as the parent of `Dictionary2` (Listing 5.6). Unlike the earlier versions, both classes now define explicit constructors for initializing their instance variables. The output of `Words2` is identical to that of the original `Words` program.

The `Dictionary2` constructor receives two integer parameters representing the number of pages and the number of definitions. Since `Book2` already has a constructor that handles the setup of the inherited `pages` variable, it makes sense to delegate that responsibility to the parent rather than duplicating the logic. Because the constructor is not inherited, it cannot be called directly — instead, the `super` reference is used to invoke it. After the parent's constructor completes, the `Dictionary2` constructor proceeds to initialize its own `definitions` variable.

In this particular example, the `pages` variable could have been set directly in `Dictionary2` without using `super`. But it is generally better practice to let each class manage its own initialization. If the `Book2` constructor were ever changed — perhaps to validate input or apply default values differently — that change would automatically propagate through any child class that properly delegates via `super`. Without delegation, corresponding changes would have to be tracked down and replicated in every subclass.

KEY CONCEPT
A parent's constructor can be invoked using the `super` reference.

A child's constructor bears the responsibility of calling its parent's constructor. As a rule, the first line of a child constructor should be a `super` call that invokes the appropriate parent constructor. If this call is absent, Java will automatically insert a no-argument `super()` call at the start of the constructor. This guarantees that the parent class fully initializes its portion of the object before the child constructor begins its own work. A `super` constructor call is only valid inside a child constructor, and it must appear as the first statement if included.

The `super` reference is not limited to constructor invocations — it can also be used to explicitly reference other variables and methods defined in the parent class, as we will see in later sections.

Multiple Inheritance

Java's model of inheritance is called single inheritance, meaning that a derived class may have exactly one parent. Some object-oriented languages extend this to allow a class to be derived from multiple parents simultaneously — a feature called multiple inheritance.

Multiple inheritance can be conceptually appealing. Suppose we have a `Car` class and a `Truck` class and want to create a `PickupTruck` class. A pickup truck shares characteristics with both — it has the passenger comfort of a car and the load-carrying capability of a truck. With single inheritance, we are forced to choose one as the parent. With multiple inheritance, `PickupTruck` could extend both simultaneously, as illustrated in Figure 8.3.

Multiple inheritance works elegantly in some scenarios, but it introduces significant complexity. What happens when both `Car` and `Truck` define methods with the same name? Which version would `PickupTruck` inherit? The resolution rules in languages that support multiple inheritance are non-trivial and can lead to subtle bugs and confusing code.

The designers of Java deliberately excluded multiple inheritance from the language. Java interfaces provide the most valuable aspects of multiple inheritance while avoiding its complications.

8.2 Overriding Methods

When a child class defines a method with the same name and signature as a method in the parent class, the child's version is said to override the parent's. The child's definition takes precedence when the method is invoked through an object of the child class. This need to override arises frequently in inheritance-based designs.

The `Messages` program in Listing 5.7 provides a straightforward illustration of method overriding. The `main` method creates two objects — one of class `Thought` and one of class `Advice` — where `Thought` is the parent of `Advice`.

Both the `Thought` class (Listing 5.8) and the `Advice` class (Listing 5.9) define a method called `message`. The version defined in `Thought` is inherited by `Advice`, but `Advice` replaces it with its own implementation. The overriding version prints a different message and then explicitly invokes the parent's version of `message` using the `super` reference, producing both outputs in sequence.

The object through which a method is invoked determines which version executes. When `message` is called through the `parked` object — an instance of `Thought` — the `Thought` version runs. When called through the `dates` object — an instance of `Advice` — the `Advice` version runs.

KEY CONCEPT
A child class can override (redefine) the parent's definition of an inherited method.

A method can be declared with the `final` modifier to prevent a subclass from overriding it. This is useful when a specific behavior must be preserved unchanged across all subclasses.

Method overriding is a cornerstone of object-oriented design. It allows classes related through inheritance to use a consistent naming convention for methods that accomplish the same general purpose in class-specific ways. Its full significance becomes especially clear in the context of polymorphism.

Shadowing Variables

While a child class can override a parent's method, it is also technically possible — though strongly discouraged — for a child class to declare a variable with the same name as one inherited from the parent. This is not the same as simply assigning a new value to the inherited variable; it is a full redeclaration of the name, producing what is called a shadow variable.

The concept is analogous to method overriding but without any of the practical benefit. Since an inherited variable is already available to the child class, redeclaring it serves no constructive purpose. Anyone reading code with shadowed variables will encounter two separate declarations that appear to apply to the same name, leading to confusion about which one is in effect at any given point. While a redeclaration could technically change the variable's type, that need almost never arises in practice. Shadowing variables introduces unnecessary ambiguity and should be avoided entirely.

```
// LISTING 5.1

//********************************************************************

// Words.java

//

// Demonstrates the use of an inherited method.

//********************************************************************

public class Words
```

```
{
   //-----------------------------------------------------------------
   // Instantiates a derived class and invokes its inherited and
   // local methods.
   //-----------------------------------------------------------------
   public static void main(String[] args)
   {
       Dictionary webster = new Dictionary();


       System.out.println("Number of pages: " + webster.getPages());
       System.out.println("Number of definitions: " +
                          webster.getDefinitions());
       System.out.println("Definitions per page: " +
                          webster.computeRatio());
   }
}
```

OUTPUT

Number of pages: 1500
Number of definitions: 52500
Definitions per page: 35.0

```
// LISTING 5.2
//********************************************************************
// Book.java
//
// Represents a book. Used as the parent of a derived class to
// demonstrate inheritance.
```

```
//*************************************************************************

public class Book

{

   protected int pages = 1500;



   //-----------------------------------------------------------------------

   // Pages mutator.

   //-----------------------------------------------------------------------

   public void setPages(int numPages)

   {

       this.pages = numPages;

   }



   //-----------------------------------------------------------------------

   // Pages accessor.

   //-----------------------------------------------------------------------

   public int getPages()

   {

       return pages;

   }

}
```

```
// LISTING 5.3

//*************************************************************************

// Dictionary.java

//

// Represents a dictionary, which is a book. Used to demonstrate
```

```java
// inheritance.

//*******************************************************************

public class Dictionary extends Book

{

   private int definitions = 52500;



   //-----------------------------------------------------------------

   // Prints a message using both local and inherited values.

   //-----------------------------------------------------------------

   public double computeRatio()

   {

       return definitions/pages;

   }



   //-----------------------------------------------------------------

   // Definitions mutator.

   //-----------------------------------------------------------------

   public void setDefinitions(int numDefinitions) {

       this.definitions = numDefinitions;

   }



   //-----------------------------------------------------------------

   // Definitions accessor.

   //-----------------------------------------------------------------

   public int getDefinitions() {

       return definitions;

   }
```

```
}
```

```
// LISTING 5.4

//********************************************************************

// Words2.java   Java Foundations

//

// Demonstrates the use of the super reference.

//********************************************************************

public class Words2

{

   //-----------------------------------------------------------------

   // Instantiates a derived class and invokes its inherited and

   // local methods.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       Dictionary2 webster = new Dictionary2(1500, 52500);


       System.out.println("Number of pages: " + webster.getPages());

       System.out.println("Number of definitions: " +

                          webster.getDefinitions());

       System.out.println("Definitions per page: " +

                          webster.computeRatio());

   }

}
```

O U T P U T

Number of pages: 1500

Number of definitions: 52500

Definitions per page: 35.0

```
// LISTING 5.5

//********************************************************************

// Book2.java

//

// Represents a book. Used as the parent of a derived class to

// demonstrate inheritance and the use of the super reference.

//********************************************************************

public class Book2

{

   protected int pages;

   //-----------------------------------------------------------------

   // Constructor: Sets up the book with the specified number of pages.

   //-----------------------------------------------------------------

   public Book2(int numPages)

   {

       pages = numPages;

   }


   //-----------------------------------------------------------------

   // Pages mutator.

   //-----------------------------------------------------------------
```

```
   public void setPages(int numPages) {

       pages = numPages;

   }


   public int getPages() {

       return pages;

   }

}
```

```
// LISTING 5.6

//********************************************************************

// Dictionary2.java

//

// Represents a dictionary, which is a book. Used to demonstrate

// the use of the super reference.

//********************************************************************

public class Dictionary2 extends Book2

{

   private int definitions;


   //-----------------------------------------------------------------

   // Constructor: Sets up the dictionary with the specified number

   // of pages and definitions.

   //-----------------------------------------------------------------

   public Dictionary2(int numPages, int numDefinitions)

   {
```

```
        super(numPages);
        definitions = numDefinitions;
    }

    //-----------------------------------------------------------------
    // Prints a message using both local and inherited values.
    //-----------------------------------------------------------------
    public double computeRatio()
    {
        return definitions/pages;
    }

    //-----------------------------------------------------------------
    // Definitions mutator.
    //-----------------------------------------------------------------
    public void setDefinitions(int numDefinitions) {
        this.definitions = numDefinitions;
    }

    public int getDefinitions() {
        return definitions;
    }
}
```

O U T P U T

I feel like I'm diagonally parked in a parallel universe.

Warning: Dates in calendar are closer than they appear.

I feel like I'm diagonally parked in a parallel universe.

```
// LISTING 5.8

//*************************************************************

// Thought.java

//

// Represents a stray thought. Used as the parent of a derived

// class to demonstrate the use of an overridden method.

//*************************************************************

public class Thought

{

   //-----------------------------------------------------------

   // Prints a message.

   //-----------------------------------------------------------

   public void message()

   {

       System.out.println("I feel like I'm diagonally parked in a " +

                          "parallel universe.");

       System.out.println();

   }

}
```

8.3 Class Hierarchies

A child class derived from one parent can itself serve as the parent of further child classes. Additionally, multiple classes can all be derived from the same single parent. As a result, inheritance relationships naturally grow into class hierarchies.

There is no upper bound on how many children a class may have, nor on how many levels a hierarchy may span. Two classes that share the same parent are called siblings. Although siblings inherit the same characteristics from their common parent, they are not themselves related by inheritance — neither is derived from the other.

KEY CONCEPT
The child of one class can be the parent of one or more other classes, creating a class hierarchy.

KEY CONCEPT
Common features should be located as high in a class hierarchy as is reasonably possible.

In a well-designed class hierarchy, common features are pushed as high as they appropriately belong. Each child class then needs only to define the characteristics that genuinely distinguish it from its parent and siblings. This approach maximizes opportunities for reuse and simplifies maintenance — a change made to a parent class propagates automatically to all of its descendants. The is-a relationship must always be preserved throughout the hierarchy as well.

Inheritance is transitive. A trait passed from a parent to its children is in turn passed from those children to their own children, and so on down the hierarchy. An inherited feature might have originated in the immediate parent or in a distant ancestor several levels above.

There is no single universally correct way to organize a class hierarchy. The structural decisions made during design shape and constrain all subsequent design and implementation choices, so they must be made thoughtfully.

The `Object` Class

KEY CONCEPT
All Java classes are derived, directly or indirectly, from the `Object` class.

In Java, every class ultimately traces its lineage back to the `Object` class. If a class definition does not use the `extends` clause to derive itself explicitly from another class, Java automatically derives it from `Object`. Therefore, these two declarations are equivalent:

class Thing

{

```
    // whatever
}
```

and:

```
class Thing extends Object
{
    // whatever
}
```

KEY CONCEPT

The `toString` and `equals` methods are inherited by every class in every Java program.

Because all classes inherit from `Object`, all of `Object`'s public methods are available through any object in any Java program. The `Object` class is defined in the `java.lang` package of the Java standard class library.

We have been relying on `Object` methods throughout the book, often without realizing it. The `toString` method, for instance, is defined in `Object`, which is why it can be called on any object whatsoever. As we have seen many times, when an object is passed to `println`, the `toString` method is invoked automatically to determine what text to display.

This means that whenever we define a `toString` method in one of our own classes, we are overriding an inherited definition from `Object`. The default implementation provided by `Object` returns a string combining the object's class name with a unique numeric identifier for that specific object — rarely what we want. We generally override it to produce something more informative and meaningful. The `String` class itself overrides `toString` so that it simply returns the stored string value.

Similarly, when we define an `equals` method in a class we are overriding an inherited method from `Object`. The version inherited from `Object` returns `true` only when the two references being compared point to the exact same object in memory — that is, when they are aliases. Classes frequently override this default definition to provide a more meaningful notion of equality. The `String` class, for example, overrides `equals` so that it returns `true` when both strings contain the same sequence of characters, regardless of whether they are the same object.

Abstract Classes

KEY CONCEPT

An abstract class cannot be instantiated. It represents a concept on which other classes can build their definitions.

An abstract class represents a generic, high-level concept within a class hierarchy. As the name suggests, it models an entity that is too broadly defined to be useful on its own. An abstract class typically provides a partial description — a shared foundation that all of its descendants will inherit and build upon. In every other respect it resembles a regular class, except that it may contain methods that have no implementation.

An abstract class cannot be instantiated, and it usually contains one or more abstract methods — methods that are declared but left without a body of code. Because an abstract method has no implementation, it cannot be invoked directly. An abstract class may also contain fully implemented non-abstract methods and regular data declarations; these behave exactly as they would in any ordinary class.

A class is marked as abstract by including the `abstract` modifier in its class header. Any class that contains at least one abstract method must be declared abstract. Each individual abstract method must also carry the `abstract` modifier. It is worth noting that a class may be declared `abstract` even if it contains no abstract methods at all — the designer may have reasons to prevent direct instantiation.

A `Vehicle` class at the top of the hierarchy may be too generic to be instantiated directly in a particular application, making it a natural candidate for an abstract class. In UML, both abstract class names and abstract method names are displayed in italics.

Concepts common to all vehicles — such as speed — can be defined in `Vehicle` and inherited by all descendants, preventing redundant and potentially inconsistent definitions from proliferating throughout the subclasses. An abstract method such as `fuelConsumption` can be declared in `Vehicle` to establish that all vehicles must support this operation and to define a consistent method signature for it, while leaving the actual computation to each subclass, which can provide an implementation tailored to its specific type.

On the other hand, concepts that apply only to some vehicles should not be placed in `Vehicle`. A variable called `numberOfWheels`, for instance, would not belong there because not all vehicles have wheels. Child classes for which that concept is relevant can introduce it at the appropriate level in the hierarchy.

There is no restriction on where in a hierarchy an abstract class may appear. They most commonly reside near the top, but it is entirely valid to derive an abstract class from a non-abstract parent.

KEY CONCEPT

A class derived from an abstract parent must override all of its parent's abstract methods, or the derived class will also be considered abstract.

A concrete child of an abstract class is expected to provide implementations for all of the abstract methods it inherits. This is simply a specific case of method overriding — supplying a definition that the parent left open. If a child class does not provide implementations for every inherited abstract method, it is itself considered abstract and cannot be instantiated.

It would be contradictory to declare a method as both `abstract` and `final`. Since a `final` method cannot be overridden in subclasses, an `abstract final` method would have no mechanism by which it could ever be given an implementation — the two modifiers are mutually exclusive. Similarly, declaring a method as both `abstract` and `static` makes no sense: a `static` method can be invoked through the class name without creating an instance, but an abstract method has no body to execute.

Choosing which classes and methods to make abstract is one of the more significant decisions in the design of a class hierarchy, and it should be made deliberately and thoughtfully. Used well, abstract classes create flexible, extensible designs that accommodate both present needs and future evolution.

8.4 Visibility

KEY CONCEPT
Private members are inherited by the child class, but cannot be referenced directly by name. They may be used indirectly, however.

As noted earlier in this chapter, all variables and methods in a parent class — including those declared `private` — are inherited by child classes. Private members exist within objects of derived classes and occupy their share of memory; the restriction is simply that they cannot be referenced directly by name in the child class. They can, however, be accessed indirectly.

The `FoodAnalyzer` program in Listing 5.10 illustrates this principle. Its `main` method creates a `Pizza` object and calls a method to calculate how many calories per serving the pizza contributes through its fat content.

The `FoodItem` class in Listing 5.11 represents a generic food item. Its constructor accepts the number of fat grams and the number of servings. The `calories` method computes the total caloric contribution of the fat, and the `caloriesPerServing` method calls `calories` internally to compute fat calories on a per-serving basis.

The `Pizza` class in Listing 5.12 extends `FoodItem` but adds no new data or behavior of its own. Its sole constructor delegates to the `FoodItem` constructor via `super`, specifying eight servings per pizza.

In the `main` method, the `special` object — a `Pizza` instance — is used to call `caloriesPerServing`, which is a `public` method inherited from `FoodItem`. Internally, `caloriesPerServing` calls `calories`, which is declared `private`. Furthermore, `calories` references `fatGrams` and the constant `CALORIES_PER_GRAM`, both also declared `private` in `FoodItem`.

The `Pizza` class has no ability to reference `calories`, `fatGrams`, or `CALORIES_PER_GRAM` directly — they are private and therefore name-inaccessible. Yet they are all fully functional within the `Pizza` object at runtime: the `Pizza` object cannot invoke `calories` directly, but it can invoke a public method that does. A `FoodItem` object is never directly created or needed for any of this to work.

```
// LISTING 5.9

//*************************************************************************

// Advice.java

//

// Represents some thoughtful advice. Used to demonstrate the use

// of an overridden method.

//*************************************************************************

public class Advice extends Thought

{

   //-----------------------------------------------------------------

   // Prints a message. This method overrides the parent's version.

   //-----------------------------------------------------------------

   public void message()

   {

       System.out.println("Warning: Dates in calendar are closer " +

                          " than they appear.");

       System.out.println();

       super.message(); // explicitly invokes the parent's version

   }

}
```

```java
// LISTING 5.10

//*****************************************************************

// FoodAnalyzer.java

//

// Demonstrates indirect access to inherited private members.

//*****************************************************************

public class FoodAnalyzer

{

   //-----------------------------------------------------------------

   // Instantiates a Pizza object and prints its calories per serving.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       Pizza special = new Pizza(275);

       System.out.println("Calories per serving: " +

                          special.caloriesPerServing());

   }

}
```

OUTPUT

Calories per serving: 309

```java
// LISTING 5.11

//*****************************************************************

// FoodItem.java

//
```

```java
// Represents an item of food. Used as the parent of a derived class
// to demonstrate indirect referencing.
//*****************************************************************
public class FoodItem
{
   final private int CALORIES_PER_GRAM = 9;
   private int fatGrams;
   protected int servings;

   //-----------------------------------------------------------------
   // Sets up this food item with the specified number of fat grams
   // and number of servings.
   //-----------------------------------------------------------------
   public FoodItem(int numFatGrams, int numServings)
   {
       fatGrams = numFatGrams;
       servings = numServings;
   }

   //-----------------------------------------------------------------
   // Computes and returns the number of calories in this food item
   // due to fat.
   //-----------------------------------------------------------------
   private int calories()
   {
       return fatGrams * CALORIES_PER_GRAM;
   }
```

```
   //-----------------------------------------------------------------
   // Computes and returns the number of fat calories per serving.
   //-----------------------------------------------------------------
   public int caloriesPerServing()
   {
       return (calories() / servings);
   }
}
```

```
// LISTING 5.12
//*****************************************************************
// Pizza.java
//
// Represents a pizza, which is a food item. Used to demonstrate
// indirect referencing through inheritance.
//*****************************************************************
public class Pizza extends FoodItem
{
   //-----------------------------------------------------------------
   // Sets up a pizza with the specified amount of fat (assumes
   // eight servings).
   //-----------------------------------------------------------------
   public Pizza(int fatGrams)
   {
```

```
        super(fatGrams, 8);

    }

}
```

8.5 Designing for Inheritance

KEY CONCEPT
Software design must carefully and specifically address inheritance.

Inheritance is one of the defining characteristics of object-oriented software, and as such it must be deliberately and specifically considered during the design process. A small investment of thought at the design stage can produce a far more elegant and maintainable architecture, with benefits that compound significantly over time.

The following guidelines summarize the key design considerations related to inheritance that should inform your decision-making throughout the program design stage:

Every derivation should represent a genuine is-a relationship — the child class should be a more specific version of the parent, not merely a convenient place to reuse code.

Design class hierarchies with reuse in mind, both for current needs and for likely future extensions. Think ahead about what new classes might eventually be introduced and whether the hierarchy as designed would accommodate them naturally.

As classes and objects are identified in the problem domain, actively look for what they have in common. Push shared features as high in the hierarchy as is appropriate, promoting consistency and reducing the maintenance burden.

Override methods wherever needed to tailor or extend inherited behavior in ways that are appropriate to the child class.

Introduce new instance variables in child classes as needed, but never shadow (redeclare) variables that are already inherited from the parent.

Let each class be responsible for managing its own data. Use the `super` reference to invoke the parent's constructor and to call overridden parent methods wherever doing so is appropriate.

Design the class hierarchy to fit the specific needs of the application, while keeping an eye on how that design might serve future needs as well.

Even when there is no immediate need, override general-purpose methods such as `toString` and `equals` in child classes where appropriate, so that inherited default implementations do not cause subtle, unintended problems down the line.

Use abstract classes to define a consistent interface that concrete classes lower in the hierarchy are required to fulfill.

Apply visibility modifiers with care, granting exactly the access that derived classes require without compromising encapsulation more than necessary.

Restricting Inheritance

KEY CONCEPT
The `final` modifier can be used to restrict inheritance.

We have used the `final` modifier extensively to declare constants. It has two additional uses in the context of inheritance that can significantly shape software design.

The first we touched on earlier in this chapter: a method declared `final` cannot be overridden in any subclass. This is used when a designer wants to guarantee that a specific behavior is preserved unchanged across all descendant classes, regardless of how those classes are otherwise specialized.

The second use of `final` applies to an entire class. A `final` class cannot be extended at all. Consider the following:

```
public final class Standards

{

   // whatever

}
```

With this declaration, the `Standards` class cannot appear in the `extends` clause of any other class — the compiler will reject any such attempt. `Standards` can be used normally as a class, and objects can be instantiated from it; it simply cannot serve as a parent.

Deciding to mark a class or method as `final` is a deliberate design choice. It is most appropriate when allowing subclasses to override or extend certain functionality would risk undermining behavior that must be handled in a specific, controlled way. This issue resurfaces in the discussion of polymorphism in Chapter 6.

Summary of Key Concepts

■ Inheritance is the process of deriving a new class from an existing one.
■ One purpose of inheritance is to reuse existing software.
■ Inheritance creates an is-a relationship between the parent and child classes.
■ Protected visibility provides the best possible encapsulation that permits inheritance.
■ A parent's constructor can be invoked using the `super` reference.
■ A child class can override (redefine) the parent's definition of an inherited method.
■ The child of one class can be the parent of one or more other classes, creating a class hierarchy.
■ Common features should be located as high in a class hierarchy as is reasonably possible.
■ All Java classes are derived, directly or indirectly, from the `Object` class.
■ The `toString` and `equals` methods are inherited by every class in every Java program.
■ An abstract class cannot be instantiated. It represents a concept on which other classes can build their definitions.
■ A class derived from an abstract parent must override all of its parent's abstract methods, or the derived class will also be considered abstract.
■ Private members are inherited by the child class, but cannot be referenced directly by name. They may be used indirectly, however.
■ Software design must carefully and specifically address inheritance.
■ The `final` modifier can be used to restrict inheritance.